

TEMA 1 – Introducción a JavaScript

🎯 Objetivo del tema

En este tema aprenderemos las bases de JavaScript

1.1 Introducción

JavaScript es un lenguaje de programación de alto nivel, interpretado o compilado *Just-In-Time* (JIT), multiparadigma (orientado a objetos basado en prototipos, imperativo y funcional), de tipado dinámico y débil, y monohilo (*single-threaded*) con un modelo de ejecución asíncrono basado en un bucle de eventos (*Event Loop*). Es el lenguaje estándar de procesamiento en el lado del cliente (*client-side*) en la web, cuya especificación oficial y estandarizada es ECMAScript (ISO/IEC 16262).

... de nuevo, una definición de ChatGPT.

JavaScript es un lenguaje de programación de alto nivel, muy parecido al Java en cuanto a sintaxis, pero que NO tiene nada que ver con Java. Utiliza Orientación a Objetos, pero el tipado de datos funciona de una forma muy laxa (débil). La gran peculiaridad del lenguaje es que su código fuente NO se compila y se ejecuta aparte, sino que se envía el código fuente directamente al Navegador, que lo compila y ejecuta al vuelo. A demás, está pensado específicamente para trabajar con eventos.

Puntos clave de JavaScript que tenemos que controlar:

- ¿Dónde coloco el código JavaScript en un HTML?
- Sintaxis (se parece a Java) y Variables (tiene algunas cosas diferentes).
- Estructuras de control: if, else, for... (se parecen a Java)
- Funciones y ámbito (parecido, pero con sus rarezas)

No vamos a hacer un estudio exhaustivo de todos los detalles que tiene JavaScript. Porque, para empezar, ya sabes programar (aprendiste en 1º DAW). Segundo, porque disponemos de Internet y un montón de recursos donde podemos consultar cualquier duda que tengamos, pedir ejemplos, etc. Por tanto, veremos ejemplos, haremos ejercicios, y aprenderás a valerte por tu cuenta.

Una web muy útil: <https://www.w3schools.com/js/>

🎯 Importante – ¿Puedo usar la IA para aprender JavaScript?

No es buena idea usar una IA para aprender a programar nada en absoluto. Y el motivo es que la usáis mal. Le pedís que os resuelva un ejercicio, lo leéis, lo probáis, y os decís “lo he entendido”. ¿Pero sabes qué? No lo has trabajado tú por tú cuenta y, por tanto, no habéis aprendido nada.

Por el contrario, sí que es cierto que la IA, acelera mucho el desarrollo de programas, la solución de errores, etcétera. ¿Y sabes por qué? Porque los programadores que la usan YA SABEN programar. Si no, no podrían sacarle el 100% a la IA.

1.2 ¿Dónde pongo el código?

A partir de este apartado, haremos referencia a una serie de ficheros ubicados en la carpeta **Ejemplos** de los apuntes. Tenlos abiertos cada vez que trabajemos un punto nuevo.

Hay **tres formas** de incluir código JavaScript en tu documento HTML. Cualquiera de las tres funciona. Un Navegador no va a dejar de hacer funcionar tu página web como debe porque uses una u otra forma. O incluso si las mezclas. Esto está heredado de una época en la que todo lo que hacías en Internet tenía que ser tolerante a fallos. Obviamente, esto complicaba mucho el mantenimiento de las webs.

- **Inclusión Externa:** El código JavaScript tiene que estar en un fichero .js **separado**. Se vincula en el HTML mediante la etiqueta `<script>`. Así tenemos las responsabilidades bien separadas.
- **Inclusión Interna:** El código JavaScript se añade directamente al HTML en las etiquetas `<script>`. No recomendable, pero puede valer para scripts mínimos y concretos.
- **Inclusión en Línea:** Escribir código JavaScript directamente en los atributos de los ejemplos. Esto está terminantemente prohibido. Dificulta el mantenimiento, rompe la separación de capas, y viola las directrices de seguridad.

```
<button onclick="alert('Hola') ">Clic</button>
```

En el Desarrollo Web moderno, se usa **un estándar estricto** en concreto. Y es el que vamos a usar. Y por supuesto, es la **Inclusión Externa**.

1. Abre los ficheros **1.Holamundo.html** y **1.Holamundo.js**
2. Lee ambos ficheros.
3. Abre **1.Holamundo.html** con el navegador.
4. Observa lo que sucede.

Lo que sucede es que el Navegador lee el fichero .html, y lo primero que se encuentra es la etiqueta `<head>` en la que le indican vincular el fichero .js. Una vez lo hace, el navegador recorre el HTML, hasta que se encuentra la etiqueta `<script>`. Esa etiqueta le ordena ejecutar el método **holamundo ()**. Este método se encuentra definido en el fichero .js. El código de ese método hace aparecer un pop-up con un mensaje de texto. Si te fijas, la página aún no se muestra: el programa está esperando a que pulses Ok. En cuanto lo haces, la página termina de renderizarse en el navegador.

Cosas con las que nos tenemos que quedar:

- Poner en `<head>` los ficheros .js
- Hacer uso de la etiqueta `<script>`
- Los métodos se definen en el .js mediante la palabra reservada **function**.
- El **alert ()** es un método propio de JavaScript que muestra pop-ups informativos.
- Se llama a los métodos por su nombre desde la etiqueta `<script>`

1.3 Salida estándar de JavaScript

Puede sonar raro, pero la salida del JavaScript no es el HTML que estás viendo por pantalla. El HTML de una página web es **estático**, nunca cambia. Otro tema diferente es que podamos **modificarlo** mediante JavaScript de forma temporal. Pero me voy por las ramas.

1. Abre el fichero **2.Instrucciones de Salida.html**
2. Lee el fichero.
3. Abre **2.Instrucciones de Salida.html** con el navegador.
4. Observa lo que sucede.

Lo que te tiene que llamar la atención, a parte de los pop-ups, es que al cargarse la página donde debería poner “Párrafo de prueba” según el HTML en realidad pone “Párrafo cambiado”. A esto nos referíamos con modificar el HTML.

Cosas con las que nos tenemos que quedar:

- **window.alert()**: muestra mensajes.
- **document.write()**: permite añadir etiquetas HTML para formatear la salida, pero no se suele usar.
- **document.getElementById ()**: Permite recuperar una **referencia** a un elemento del HTML. Si la combinamos con la instrucción **innerHTML**, Esto te permite poder manipular **los nodos hijos** de ese elemento. En el ejemplo, cambiamos el texto; pero es posible hacer otras cosas como añadir un elemento nuevo a una lista .
- **console.log()**: permite lanzar trazas de texto en la consola del DevTools para debugear.

1.4 Entrada estándar de JavaScript

JavaScript incluye tres métodos nativos de la interfaz window para mostrar cuadros de diálogo simples. Todos ellos tienen en común que **son modales, bloquean la ejecución** del código, y no permiten personalización mediante CSS. Uno de ellos ya lo has visto: es el window.alert(). Para pedir una entrada al usuario, se usan los otros dos, que son:

- **window.confirm()**: muestra una pregunta con dos botones: "Aceptar" y "Cancelar". Retorna un valor booleano para poder diferenciar la selección del usuario.
- **window.prompt()**: muestra un campo para pedir texto con botones "Aceptar" y "Cancelar". Retorna un String o un null con los datos introducidos por el usuario.

Importante – ¿Y todo esto se usa?

De lo que hemos visto hasta el momento, en el mundo profesional, quizás solamente el getElementById(). Lo demás o no se usa, o solamente durante pruebas, o hay mejores alternativas. Pero son imprescindibles para aprender JavaScript.

1.5 Sintaxis

La Sintaxis de JavaScript se parece mucho a la de Java. Tanto, que si sabes Java podrías leer y entender el código JavaScript. Ahora bien, tiene diferencias. En este apartado, vamos a ver cómo es esa Sintaxis de JavaScript haciendo hincapié en esos detalles.

1. Abre el fichero **3. Sintaxis JavaScript.html**
2. Lee el fichero.
3. Abre **3. Sintaxis JavaScript.html** con el navegador.
4. Observa lo que sucede.

Lo interesante de este fichero no es la página en el navegador. Observa en el HTML. Nos está haciendo un magnífico mini resumen de la sintaxis más elemental de JavaScript.

Cosas con las que nos tenemos que quedar:

- Para declarar una variable, se usa **var**.
- Puedo hacer sumas de variables y cosas similares como en Java.
- El nombre de las variables puede llevar \$ y _ por lo demás, igual que Java.

Sigamos con más cosas obvias:

1. Abre el fichero **4. Sentencias en Javascript.html**
2. Lee el fichero.
3. Abre **4. Sentencias en Javascript.html** con el navegador.
4. Observa lo que sucede.

De nuevo, mira el HTML. El fichero es un pequeño resumen de cómo se puede escribir tu código JavaScript.

Cosas con las que nos tenemos que quedar:

- ¿Te has dado cuenta de que cuando declaras una variable con **var**, no se dice el tipo de dato?
- Los ámbitos (bloques de código) se declaran mediante { }

Aún más obviedades:

1. Abre el fichero **5. Comentarios en JavaScript.html**
2. Lee el fichero.
3. Abre **5. Comentarios en JavaScript.html** con el navegador.
4. Observa lo que sucede.

Los comentarios en JavaScript se escriben igual que en Java. A producción nunca se suben comentarios, ni mucho menos en unos ficheros .js que viajan como texto plano por la red. Esto podría revelar inadvertidamente fallos de seguridad. Dicho esto, estás aprendiendo, pon todos los comentarios que necesites.

1.6 Sintaxis: Variables

Empezamos con las diferencias respecto a Java. Acostumbrado a tener que especificar constantemente el tipo de dato de una **variable**, descubrirás que en JavaScript no es necesario. Pero tiene truco.

JavaScript es un lenguaje de **tipado dinámico y débil**, lo que significa que no se declara el tipo de dato al definir la variable y las variables pueden cambiar de tipo en tiempo de ejecución. Esto significa que una variable no tiene tipo cuando la declaras, gana un tipo cuando le asignas un valor, pero si le asignas otro diferente de otro tipo, no da error.

Esto es una fuente interminable de problemas. Ya te irás dando cuenta.

JavaScript define dos grupos diferentes de tipos de datos.

- **Tipos primitivos:** Inmutables, pasados por valor, similares a Java. Una vez asignado un valor a un tipo primitivo, **NO puede ser cambiado**, cualquier intento genera copias.
 - Number
 - BigInt
 - String
 - Boolean
 - Undefined: es el valor que adquiere una variable automáticamente cuando no la asignas un valor. Suene aparecer como error cuando estás aprendiendo. No es lo mismo que Null.
 - Null
 - Symbol
- **Tipos complejos:** Mutables, pasados por referencia, te sonarán de Java. Permiten cambios de valores.
 - Object
 - Array
 - **Function** ← Si puedes meter funciones en una variable. Ya lo veremos.
 - Date, Map, Set, RegExp, etc.

Para los ejercicios simples que proponemos más adelante, es posible asignar valores a una variable y no preocuparnos de nada más. No obstante, pueden aparecer problemas dado que JavaScript hace **Coerción de Tipos**. Es decir, hace conversiones de tipos de forma automática sin preguntar a nadie.

- Si intentas hacer **n = "5" + 2**, es decir, hacer **n = String + Number**, JavaScript no te va a dar ningún error. Eso sí, lo que hará es concatenar "52" y hacer que N sea un String.
- Si intentas hacer **n = "5" - 2**, en cambio, lo que hace es una operación matemática y daría N = 3.
- Si intentas hacer **n = false + 1**, el resultado es 1, porque false se convierte en 0, true en 1
- Etc....

La mayoría de los programadores de JavaScript veteranos se saben todos por la práctica. Tú tenlo en cuenta cuando te den errores extraños al hacer operaciones de cálculo.

Es posible comprobar con el operador **typeof** el tipo de dato de una variable, o forzar un **casting** explícito de una variable a un tipo de datos (Number ("123") te devuelve un Number).

Vamos a los ejemplos para ir mirando cositas:

1. Abre el fichero **6. Variables en JavaScript.html**
2. Lee el fichero.
3. Abre **6. Variables en JavaScript.html** con el navegador.
4. Observa lo que sucede.

Cosas con las que nos tenemos que quedar:

- La variable **nombre** es definida, pero no se le asigna valor. Por eso se escribe **undefined** en el primer párrafo. Eso no ocurre en el segundo párrafo. Recuerda que undefined NO es lo mismo que null.
- Observa que en el tercer párrafo opero con Number y Strings sin preocuparme del tipo.

Veamos ahora la asignación de valores a variables:

1. Abre el fichero **7. Operadores aritméticos.html**
2. Lee el fichero.
3. Abre **7. Operadores aritméticos.html** con el navegador.
4. Observa lo que sucede.

Nos debería de sonar todo de Java.

Ejemplo de la asignación de valores a variables:

1. Abre el fichero **8. Operadores de asignación en JavaScript.html**
2. Lee el fichero.
3. Abre **8. Operadores de asignación en JavaScript.html** con el navegador.
4. Observa lo que sucede.

Nos debería de sonar todo de Java.

Ámbito de una variable

Un error muy típico en JavaScript cuando tu código se vuelve largo y complicado, es tener problemas con el **ámbito** de una variable. En JavaScript existen dos ámbitos bien definidos

- **Ámbito Global:** una variable definida **fuera** de una función está disponible para todo el programa. Ignora absolutamente todas las {...} que encuentra.

```
// Ámbito Global
var mensaje = "Hola mundo";

function mostrarMensaje() {
  console.log(mensaje); // Accesible
}

mostrarMensaje();
console.log(mensaje);    // Accesible
```

- **Ámbito de Función:** esa variable que has definido solo existe dentro de esa función.

```
// Ámbito de Función
function calcular() {
  var resultado = 42;
  console.log(resultado); // Accesible
}

calcular();
console.log(resultado); // NO Accesible
```

Hay un problema con las variables tipo **var** en JavaScript. Al ignorar las {...} puede ser que cometas errores. Para solucionarlo, se introdujeron en el **JavaScript moderno** dos palabras clave nuevas: **let** y **const**. Ambas tienen como ámbito exclusivo las {...}.

- **Let:** permite reasignar un nuevo valor a la variable después de declararla. Se usa mucho en las estructuras repetitivas (for, while...)
- **Const:** significa constante. No se puede reasignar el valor ni volver a declarar la variable.

1.7 Sintaxis: Estructuras de control

Al igual que Java, JavaScript tiene estructuras de control y repetitivas. No hay mucho que decir de ellas.

Ejemplos de las estructuras.

1. Abre el fichero **9. Estructuras condicionales y repetitivas.html**
2. Lee el fichero.
3. Abre **9. Estructuras condicionales y repetitivas.html** con el navegador.
4. Observa lo que sucede.

Nos debería de sonar todo de Java.

Ejercicio 1 – Entradas y salidas

Realiza una página web que muestre tu nombre, tu primer apellido y tu segundo apellido. Para completa el ejercicio, recuerda utilizar la **Inclusión Externa** del código JavaScript. Recuerda que:

- **HTML:** Define la estructura del documento.
- **CSS:** Define los estilos.
- **JavaScript:** Define la lógica.

Pasos:

1. Solicita mediante **window.prompt()** la información necesaria.
2. Guarda el nombre y los apellidos en variables.
3. Muestra los tres datos en la página.

Ejercicio 2 – Cuadrado (for)

Realiza una página web que muestre en una matriz de 5 x 10, una sucesión de números pares. Recuerda seguir los requisitos del ejercicio anterior.

Pasos:

1. Utiliza dos bucles for, uno dentro del otro, para generar la matriz.
2. Utiliza `document.write("<br>")` para los saltos de línea.

Ejercicio 3 – Cuadrado (while)

Realiza una página web que muestre en una matriz de 5 x 10, una sucesión de números pares. Recuerda seguir los requisitos del ejercicio anterior.

Ejercicio 4 – Sucesión (for)

Realiza una página web que muestre en una matriz de 10 x 10, una sucesión de números. Comenzando por el número 1, el siguiente número de la sucesión debe ser el número más el número de veces que ya has hecho este cálculo. Es decir: 1, 3, 6, 10, 15, 21....

Ejercicio 5 – Sucesión (while)

Realiza una página web que muestre en una matriz de 10 x 10, una sucesión de números. Comenzando por el número 1, el siguiente número de la sucesión debe ser el número más el número de veces que ya has hecho este cálculo. Es decir: 1, 3, 6, 10, 15, 21....

Ejercicio 6 – ¿Desea continuar?

Realiza una página web que muestre en una matriz de 5 x 5 los números pares. A continuación, pregunta al usuario si desea continuar. Si dice que sí, dibuja los siguientes números pares en una matriz de 5 x 5. Repite el proceso hasta que el usuario dice que no desea continuar.

Ejercicio 7 – ¿Desea continuar?

Realiza una página web que muestre en una matriz de 4 columnas de números. Debes mostrar la misma sucesión que en los ejercicios 4 y 5, pero si el número a mostrar es un par, NO debes mostrarlo.

Para averiguar si un número es par o impar, divídelo por 2 y mira si su resto es 0.

Ejercicio 8 – Factorial

Realiza una página web que solicite un número. Si lo introducido no es un número o no se ha introducido nada, se debe volver a solicitar el número. Se debe multiplicar el número por todos los números anteriores a ese. Por ejemplo:

- 1- Resultado: 1
- 2- Resultado: $2 \times 1 = 2$
- 3- Resultado: $3 \times 2 \times 1 = 6$
- 4- Resultado: $4 \times 3 \times 2 \times 1 = 24$

Etc.

.